

Redrob TrustScore

A verified, evidence-based candidate ranking agent

Track 1 · Data & AI Challenge: Intelligent Candidate Discovery

Team Name : **Arcane009**

Team Leader : **Abhinav Khatta**

Team Members : **Divyanshu Kapariya, Abhinav Khatta**

Problem Statement :

Rank the top 100 candidates for a Senior AI Engineer role from a 100,000-candidate pool seeded with keyword-stuffers, hidden gems and ~80 impossible 'honeypot' profiles — without keyword matching, on CPU only, in under 5 minutes.

What is our proposed solution, and what makes it different?

- **An autonomous, rule-based ranking agent — 'TrustScore' — that scores every candidate on verified evidence of what they actually did, not on the keywords they listed.**
- For each candidate it combines seven signals into one auditable score, then returns a ranked top-100 with a plain-language reason for each pick.
- **What makes it different from traditional candidate matching:**
- Traditional systems embed the profile and rank by similarity — which rewards anyone who stuffs the right keywords. We deliberately distrust the skills list.
- We read career-history TEXT for proof of retrieval/ranking/ML-systems work, apply the JD's stated disqualifiers as penalties, and weight whether the person is even reachable.
- Every score breaks into named parts, so a recruiter can see and challenge exactly why a candidate ranks where they do — no black box.
- It runs in ~17 seconds for 100K candidates on a CPU with no network — production-realistic, not a benchmark toy.

Key requirements extracted from the JD, and the signals that decide fit.

- **Key requirements pulled from the JD:**
- Real production experience in embeddings/retrieval, vector search, ranking and evaluation (NDCG/MRR) — at PRODUCT companies, not pure research or services.
- 5–9 years experience; strong Python; based in / open to Indian metros (Pune, Noida).
- Explicit disqualifiers: title-chasers, consulting-only careers, research-only, CV/speech-only without NLP/IR.
- **Signals that matter most (fit beyond keyword matching):**
- Evidence in career descriptions — 'built a learning-to-rank model', 'RAG in production' — outweighs any skills tag.
- Actual job title and company type (product vs services) vs. a polished-but-fake skills list.
- Behavioral availability: recent login, recruiter response rate, open-to-work — a perfect-on-paper ghost is not hireable.
- Profile internal consistency — impossible timelines flag fakes/honeypots.

Ranking Methodology

Redrob TrustScore · Intelligent Candidate Discovery & Ranking

4 / 10

How candidates are retrieved, scored, and combined into a final rank.

Role fit	Is the actual role on the AI/ML/search bullseye?	weight 0.40
Evidence	Retrieval/ranking/ML work found in career TEXT	weight 0.32
Experience	5–9 year sweet spot, decaying outside	weight 0.16
Location	India metro / Pune / Noida	weight 0.12
Disqualifiers	services-only, job-hopping, research-only, CV-only	– penalty
Availability	recent login, response rate, open-to-work	× 0.55–1.10
Trust / sanity	honeypot consistency checks	× kill-switch

score = (0.40·role + 0.32·evidence + 0.16·exp + 0.12·loc – penalties) × availability

honeypots are multiplied by 0.001 → they sink below the top 100.

Heuristic / rule-based (no GPU, no LLM at runtime). Ties broken deterministically by candidate_id.

Final scores are spread into a clean [0.55, 0.99] band so the model visibly differentiates.

How decisions are explained, and how we keep them honest.

- **Every ranking decision is explained from real data:**
- Each candidate gets a 1–2 sentence reason built only from fields in their profile — title, years, evidence words actually present, recruiter response rate.
- Honest concerns are surfaced too (e.g. '120d notice', 'based outside India') so reasoning matches the rank.
- **Preventing hallucination / unsupported justifications:**
- Reasoning is generated by templating REAL field values — it cannot invent skills or employers the candidate doesn't have.
- Because the score itself is a transparent sum of named components, any claim is traceable to a number.
- **Handling inconsistent, low-quality, or suspicious profiles:**
- A honeypot kill-switch flags internally-impossible profiles (tenure > experience, expert skill used 0 months, skill used longer than the career, claims more experience than time elapsed) and pushes them out of the top 100.
- QA verified: 0 honeypots and 0 keyword-stuffers in our top 100.

End-to-End Workflow

Redrob TrustScore · Intelligent Candidate Discovery & Ranking

6 / 10

From JD input to ranked candidate output.



Single command: `python rank.py --candidates ./data/candidates.jsonl --out ./submission`

End-to-end runtime: ~17 seconds on CPU, no network.

Layered, dependency-light, reproducible.

INPUT LAYER

candidates.jsonl (100K profiles) + Job Description requirements



FEATURE / EVIDENCE EXTRACTION

career-history text · titles · skills · redrob behavioral signals



SCORING ENGINE (rule-based)

role · evidence · experience · location · penalties · availability · honeypot



RANKER

deterministic sort → top 100 → fact-grounded reasoning



OUTPUT + QA

submission.csv / submission.xlsx · qa.py validates format & traps

CPU-only · no GPU · no network · pure Python + openpyxl · ~17s / 100K · <16GB RAM.

Results & Performance

Redrob TrustScore · Intelligent Candidate Discovery & Ranking

8 / 10

Evidence the ranking is high-quality and within compute limits.

100,000

candidates ranked

~17 s

runtime on CPU

0

honeypots in top 100

0

keyword-stuffers in top 100

- **Top-100 is 100% relevant roles: ML/AI/NLP/Search/Recommendation engineers — no distractor titles.**
- Manual spot-checks confirm reasoning matches each real profile (no hallucination).
- Compute constraints met with large margin: 17s vs 5-min budget, CPU-only, zero network calls, <16GB RAM.
- Format validated: exactly 100 unique ranks & IDs, all IDs exist, scores non-increasing.

Technologies Used

Chosen for speed, reproducibility, and zero runtime dependencies.

- **Python 3 (standard library)** — JSON parsing, scoring, CSV output. No heavy ML stack needed; keeps runtime in seconds and fully reproducible.
- openpyxl — to also emit the XLSX output the portal requests.
- Pure rule-based scoring — deliberately NO LLM / GPU at runtime, so it meets the 5-min CPU, no-network constraint and scales to a 200K production pool.
- Git / GitHub — versioned, reproducible repo with a single reproduce command.
- Why this stack: the challenge rewards latency-quality tradeoffs and reproducibility. A transparent heuristic beats a black-box embedding model here — it's faster, explainable, and immune to keyword-stuffer traps.
- AI tools (Claude) were used for code review and drafting — declared honestly; no candidate data sent to any LLM, no LLM in the ranking path.

Everything needed to reproduce and verify our submission.

- **GitHub repository:** <https://github.com/Abhinav09-bits/redrob-trustscore>
- Reproduce command: `python rank.py --candidates ./data/candidates.jsonl --out ./submission`
- Ranked output: `submission.xlsx` (and `submission.csv`) — top 100 with reasoning.
- QA script: `qa.py` — verifies valid format and zero traps in the top 100.
- Sandbox / demo link: <https://redrob-trustscore-gba7ntzmq5cwxtkp6ghgkn.streamlit.app/>
- Metadata: `submission_metadata.yaml` (team, compute env, honest AI-tools declaration).